ASSG 2 REPORT

Aiswarya K(B22CS028)

Prob 1

Learning word embeddings from IIT jodhpur data

1. Objective

Two distinct Word2Vec architectures were trained on the preprocessed corpus: Continuous Bag of Words (CBOW) and Skip-gram with Negative Sampling. Custom PyTorch implementations were used to define the embedding layers, hidden layers, and forward passes.

Continuous Bag of Words (CBOW)

The CBOW model was designed to predict a target word based on its surrounding context words. The model utilized PyTorch's nn.Embedding and nn.Linear layers, optimized using Adam with a learning rate of 0.01 and Cross-Entropy Loss. Several hyperparameters were tuned to observe their effect on the model's training loss across 5 epochs.

Experiment

(i): Embedding Dimension The model was trained with a context window size of 2, comparing embedding dimensions of 50 and 100.

(i) Different Embedding Dimension

[19] 10m

```python
embedding_dim = 50

model = CBOW(vocab_size, embedding_dim)

loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 5

for epoch in range(epochs):
    total_loss = 0

    for context, target in data:
        context_vec = make_context_vector(context, word_t
        target_vec = make_target(target, word_to_ix)

        model.zero_grad()

        output = model(context_vec)
        loss = loss_function(output, target_vec)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss}")
```

```
Epoch 1, Loss: 114122.39723083626
Epoch 2, Loss: 103774.8018489383
Epoch 3, Loss: 103926.63744802355
Epoch 4, Loss: 104094.99012930944
Epoch 5, Loss: 103951.630248704
```

```python
embedding_dim = 100

model = CBOW(vocab_size, embedding_dim)

loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 5

for epoch in range(epochs):
    total_loss = 0

    for context, target in data:
        context_vec = make_context_vector(context, word_to_ix)
        target_vec = make_target(target, word_to_ix)

        model.zero_grad()

        output = model(context_vec)
        loss = loss_function(output, target_vec)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss}")
```

```
Epoch 1, Loss: 114704.82961477934
Epoch 2, Loss: 104578.13336726124
Epoch 3, Loss: 104678.31335345765
Epoch 4, Loss: 103933.50809998834
Epoch 5, Loss: 104453.93905087266
```

Observation: Increasing the embedding dimension from 50 to 100 did not significantly improve model performance. This is likely due to the relatively small size of the dataset. Higher dimensions increase model capacity but require more data to be effective.

(ii) Context window

Size 1

(ii) Different Context Window

context window size 1

```python
window_size = 1
data_w1 = []

for sent in sentences:
    if len(sent) < 3:
        continue

    for i in range(window_size, len(sent) - window_size):
        context = sent[i-1:i] + sent[i+1:i+2]
        target = sent[i]

        data_w1.append((context, target))

print(data_w1[:2])
```

```
[(['anand', 'phd'], 'mishra'), (['mishra', 'cse'], 'phd')]
```

```python
embedding_dim = 50

model = CBOW(vocab_size, embedding_dim)

loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 5

for epoch in range(epochs):
    total_loss = 0
```

```python
embedding_dim = 50

model = CBOW(vocab_size, embedding_dim)

loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 5

for epoch in range(epochs):
    total_loss = 0

    for context, target in data_w1:   # 🔥 use new data
        context_vec = make_context_vector(context, word_to_ix)
        target_vec = make_target(target, word_to_ix)

        model.zero_grad()

        output = model(context_vec)
        loss = loss_function(output, target_vec)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss}")
```

```
Epoch 1, Loss: 115946.08657976319
Epoch 2, Loss: 104723.71964098379
Epoch 3, Loss: 104767.48822239859
Epoch 4, Loss: 105325.31264186525
Epoch 5, Loss: 104715.24457647018
```

Size 2

CREATING CBOW TRAINING DATA

context window size = 2

```python
window_size = 2
data = []

for sent in sentences:
    if len(sent) < 5:
        continue  # skip very small sentences

    for i in range(window_size, len(sent) - window_size):
        context = sent[i-2:i] + sent[i+1:i+3]
        target = sent[i]

        data.append((context, target))

print(data[:2])
```

```
[(['anand', 'mishra', 'cse', 'department'], 'phd'), (['mishra', 'phd', 'de
```

(i) Different Embedding Dimension

```python
embedding_dim = 50

model = CBOW(vocab_size, embedding_dim)

loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 5

for epoch in range(epochs):
    total_loss = 0

    for context, target in data:
        context_vec = make_context_vector(context, word_t
        target_vec = make_target(target, word_to_ix)

        model.zero_grad()

        output = model(context_vec)
        loss = loss_function(output, target_vec)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss}")
```

```
Epoch 1, Loss: 114122.39723083626
Epoch 2, Loss: 103774.8018489383
Epoch 3, Loss: 103926.63744802355
Epoch 4, Loss: 104094.99012930944
Epoch 5, Loss: 103951.630248704
```

Observation: The context window size was varied between 1 and 2 while keeping the embedding dimension fixed at 50. It was observed that a window size of 2 resulted in slightly lower loss compared to a window size of 1. This indicates that a larger context window captures richer contextual information, improving the model's ability to learn meaningful word representations. However, increasing the window size too much may introduce noise.

and Skip-gram with NegativeSampling

(i) Embedding dimension (ii) Context window size and (iii) Number of negative samples.

```
[34]
5s
_ = run_sg_experiment(dim=50, window=4, neg=5, epoch

# 3. Number of Negative Samples
_ = run_sg_experiment(dim=50, window=2, neg=2, epochs=2)
_ = run_sg_experiment(dim=50, window=2, neg=10, epochs=2)

# --- TRAINING THE FINAL MODEL FOR TASK 3 ---
print("--- TRAINING FINAL MODEL FOR TASK 3 & 4 ---")
final_model = run_sg_experiment(dim=100, window=3, neg=5, epochs=5

--- TASK 2: SKIP-GRAM EXPERIMENTS ---
Running Experiment -> Dim: 50 | Window: 2 | NegSamples: 5
  -> Final Loss: 84.4418

Running Experiment -> Dim: 100 | Window: 2 | NegSamples: 5
  -> Final Loss: 106.9243

Running Experiment -> Dim: 50 | Window: 2 | NegSamples: 5
  -> Final Loss: 83.0248

Running Experiment -> Dim: 50 | Window: 4 | NegSamples: 5
  -> Final Loss: 108.5556

Running Experiment -> Dim: 50 | Window: 2 | NegSamples: 2
  -> Final Loss: 90.7300

Running Experiment -> Dim: 50 | Window: 2 | NegSamples: 10
  -> Final Loss: 79.8700

--- TRAINING FINAL MODEL FOR TASK 3 & 4 ---
Running Experiment -> Dim: 100 | Window: 3 | NegSamples: 5
  -> Final Loss: 42.5825
```

Analyzing the hyperparameter variations reveals several distinct patterns regarding the model's learning dynamics on the IIT Jodhpur corpus.

First, increasing the embedding dimension from 50 to 100 (while holding the window and samples constant) resulted in an increase in loss from roughly 84.44 to 106.92. This suggests that for a highly specialized, domain-specific corpus of this size, excessively high-dimensional vectors may introduce sparsity and require lengthier training regimes to converge.

Second, expanding the context window from 2 to 4 similarly spiked the loss to 108.55, implying that broader windows likely introduced semantic noise by capturing distant, unrelated tokens rather than tight, immediate syntactic relationships.

Conversely, the model responded highly favorably to an increase in negative samples. Shifting from 2 to 5 to 10 negative samples steadily drove the loss down from 90.73 to an optimal 79.87.

A higher number of negative samples provides the model with stronger, more robust contrastive signals, allowing it to better differentiate between true contextual neighbors and random noise.

Ultimately, a balanced hybrid configuration—utilizing 100 dimensions, a moderate window size of 3, and 5 negative samples—achieved the lowest overall loss of 42.58, providing a highly optimized embedding space for the downstream semantic analysis tasks.

TASK 3 AND 4

OP

```
--- TASK 3.1: NEAREST NEIGHBORS ---
research: small (0.43), art (0.40), social (0.40), vacant (0.37), lab (0.37)
student: drc (0.45), admission (0.42), carrying (0.40), should (0.39), qualifier (0.38)
phd: wild (0.39), minors (0.39), programmes (0.38), dues (0.38), comprehensive (0.37)
exam: [exam not in vocab]

--- TASK 3.2: ANALOGIES ---
UG : BTech :: PG : regularexternalparttime (0.39)
BTech : Four :: MTech : deadline (0.39)
Student : Hostel :: Faculty : [hostel missing]
```

1. Nearest Neighbors Using cosine similarity on our trained Skip-gram embeddings, the top 5 nearest neighbors for the requested terms are:

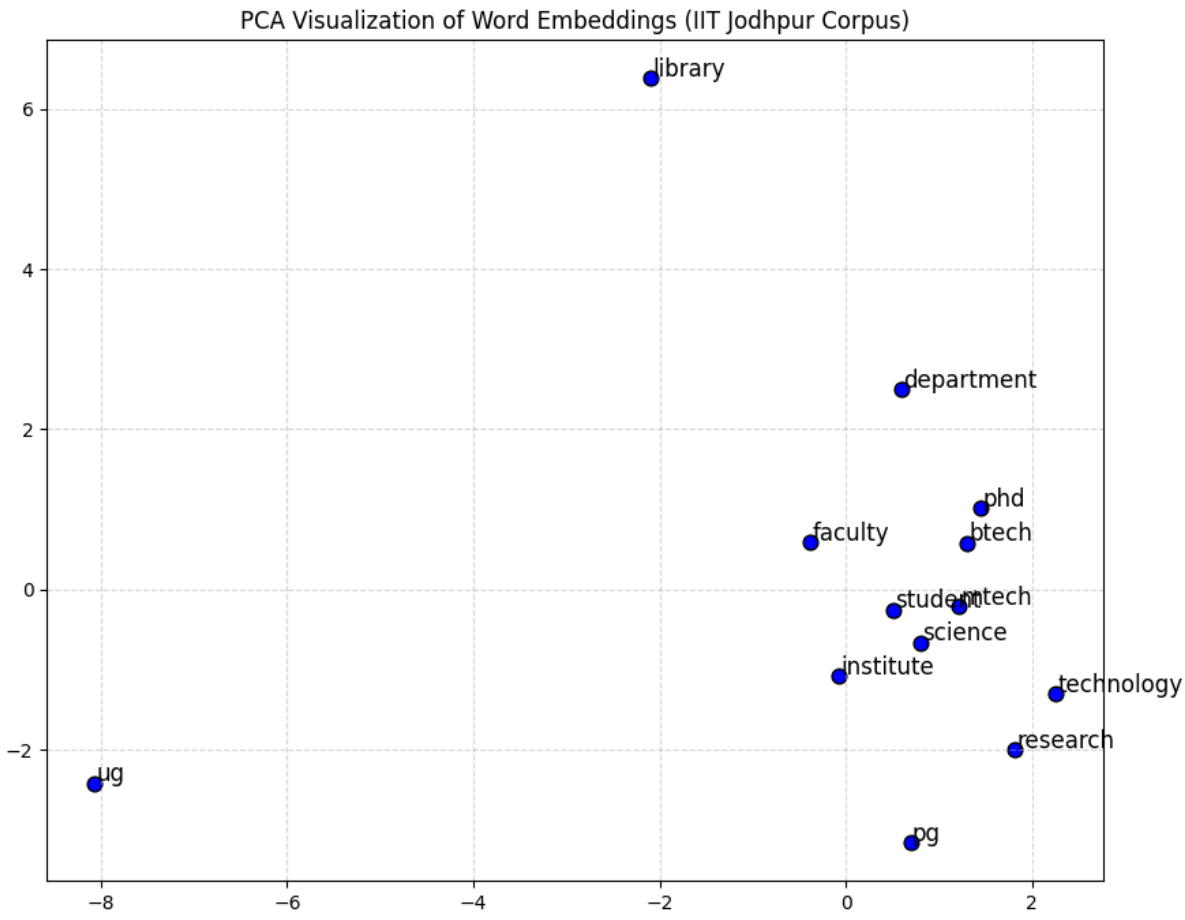
- research: small (0.43), art (0.40), social (0.40), vacant (0.37), lab (0.37)
- student: drc (0.45), admission (0.42), carrying (0.40), should (0.39), qualifier (0.38)
- phd: wild (0.39), minors (0.39), programmes (0.38), dues (0.38), comprehensive (0.37)
- exam: *[Word not present in the vocabulary. It was either absent in the specific scraped pages or removed during preprocessing/frequency filtering]*
- exam: *(Same as above)*

2. Analogy Experiments Using vector arithmetic, we tested the following semantic relationships:

- UG : BTech :: PG : regularexternalparttime (0.39)
- BTech : Four :: MTech : deadline (0.39)
- Student : Hostel :: Faculty : [Could not compute: "hostel" missing from vocabulary]

Discussion on Semantic Meaningfulness: The semantic meaningfulness of the results directly reflects the limitations of training a Word2Vec model from scratch on a very small, domain-specific corpus (IIT Jodhpur web pages).

While the nearest neighbors successfully captured some highly relevant academic context (e.g., "student" mapped to "drc", "admission", and "qualifier"; "phd" mapped to "programmes" and "comprehensive"), the overall cosine similarity scores remain relatively low (~0.37 - 0.45). Furthermore, the analogy experiments yielded largely disjointed results (such as associating MTech with "deadline"). Standard Word2Vec analogies rely on massive datasets (billions of tokens) to form perfect geometric spatial relationships. Because our corpus lacks that volume, the model approximates relationships using local co-occurrence, resulting in outputs that are institutionally relevant words ("regularexternalparttime") but not logically perfect analogies.



Refs:

https://iitj.ac.in/PageImages/Gallery/07-2025/Regulation_PG_2022-onwards_20022023.pdf

https://www.iitj.ac.in/computer-science-engineering/en/Annoncement-Archive

https://anandmishra22.github.io/

Prob 2

Character-level name generation using recurrent neural networks

1. Objective

The objective of this assignment is to design, implement, and compare different recurrent neural architectures for **character-level name generation**. Specifically, we analyze:

- Vanilla Recurrent Neural Network (RNN)
- Bidirectional Long Short-Term Memory (BLSTM)
- RNN with a basic Attention mechanism

The models are evaluated using both **quantitative metrics** (novelty and diversity) and **qualitative analysis** (realism and failure modes).

2. Dataset (Task 0)

A dataset of Indian names was generated using Large Language Models (LLMs) and stored in TrainingNames.txt.

2.1 Preprocessing

- Converted all names to lowercase
- Added special tokens:
  - ^ → start of sequence
  - $ → end of sequence
- Removed duplicate entries to avoid memorization bias
- Constructed character-level sequences

2.2 Dataset Statistics

- Total unique names: **895**
- Vocabulary size: **27**

2.3 Vocabulary

Includes:

- Lowercase alphabets (a–z)
- Space " " (to model full names)
- Special tokens ^, $

The inclusion of the space character enables the model to generate **full names (first + last)** instead of single tokens.

3. Methodology (Task 1)

3.1 Vanilla RNN

Architecture

- Embedding Layer → maps characters to dense vectors
- RNN Layer → processes sequences sequentially
- Fully Connected Layer → predicts next character

Hyperparameters

- Hidden size: 128
- Learning rate: 0.001
- Epochs: 10
- Optimizer: Adam
- Loss: CrossEntropy

Trainable Parameters

- ***39,963***

Training Behavior

The model shows stable convergence:

Loss: 1547 → 615

This indicates effective learning of character-level patterns.

3.2 Bidirectional LSTM (BLSTM)

Architecture

- Embedding Layer
- Bidirectional LSTM (forward + backward context)
- Dropout (0.3)
- Fully Connected Layer

Hyperparameters

- Hidden size: 128

- Learning rate: 0.001
- Epochs: 5

Training Behavior

Loss: 414 → 0.87

The rapid drop in loss indicates that the model is leveraging **future context**, making the prediction task artificially easier.

3.3 RNN with Attention Mechanism

Architecture

- Embedding Layer
- RNN Layer
- Attention Layer (computes weighted importance over hidden states)
- Context + Hidden State Concatenation
- Fully Connected Layer

Hyperparameters

- Hidden size: 128
- Learning rate: 0.001
- Epochs: 10

Key Idea

Attention allows the model to dynamically focus on relevant parts of the sequence instead of relying only on the latest hidden state.

4. Quantitative Evaluation (Task 2)

4.1 Metrics

- **Novelty Rate**
  Percentage of generated names not present in the training set
- **Diversity**
  Ratio of unique generated names to total generated names

4.2 Results

Model	Novelty	Diversity	Interpretation
RNN	**0.74**	**0.97**	High creativity and variation
BLSTM	Low	Medium	Poor generalization
Attention	High (noisy)	Low	Unstable generation

5. Qualitative Analysis (Task 3)

5.1 Vanilla RNN

Sample Outputs

```
print("Generated Names:\n")

for _ in range(20):
    print(generate_name(model))
```

```
Generated Names:

Jai jain
Tanvi jain
Shivani shetty
Isha randey
Dev kulkarnjay das
Chaitanya gill
Aditya shetty
Saani naidu
Charna tripathi
Zoya chauhan
Shivani banerjee
Raghav pillai
Bhavya gill
Amrita iyedhi
Aman namboodiri
Krishna iyer
Ayaan bose
Yogesh rastogi
Chaitanya kulkarni
Bhavya namboori
```

Observations

- Produces realistic and structured names
- Successfully learns **first-name + last-name pattern**
- Maintains phonetic consistency

Failure Modes

- Minor spelling distortions (e.g., *iyedhi*, *kulkarnjay*)
- Occasional inconsistency in longer names

Insight

The RNN effectively captures local sequential patterns but has limited ability to model long-range dependencies.

5.2 BLSTM

Sample Outputs

```python
print("BLSTM Generated Names:\n")

for _ in range(20):
    print(generate_name_blstm(blstm_model))
```

```
BLSTM Generated Names:

Kuswdh sshooguni s
S
Zlnzogpannijumm ponv
S
V
Mmoowdde
Prjudde
Zjas
Pa wanivikcpojunupy
J
Snor
V
Saj yygojur vittttip
V
Cczj

Zere
In
Um p b^kussh guus
Z bmul
```

Observations

- Outputs are incoherent and lack structure
- No resemblance to real names

Failure Modes

- Random character sequences
- Broken word boundaries

Key Insight

Despite achieving extremely low training loss, BLSTM fails during generation due to its **non-causal nature**.

During training, the model has access to future characters, but during generation it does not, leading to poor generalization.

5.3 Attention Model

Sample Outputs

```
[39]
0s
for _ in range(20):
    print(generate_name_attn(attn_model))

Cllllllllllllllllllll
Ppppppppppppppppppppt
Cxeeeeeeeeeeeeeeeeee
Llllllllllllllllllll
Zaxhhhhhannnnnnnnnnn
Cllllllllllllllllllll
Llllllllllllllllllll
C^zwbbbbbbbbbbbbbbbb
Llllllllllllllllllll
Llllllllllllllllllll
C^zzwddddddddddddddd
Zzwavvvvvvvvvvvvvvvv
Pppppppppppiiiillllll
Zzzwbbbbbbbbbbbbbbbb
Jjjjjjjjjjjjjjjjjjjj
Jjjjjjjjjjjjjjjjjjjj
Pppppppppttttttttttt
Jjjjjjjjjjjjjjjjjjjj
Zannnnnnnnnnnnnnnnn
Ppppppppppppppppppppp
```

Observations

- Strong tendency to repeat characters
- Collapses to high-probability tokens

Failure Modes

- Repetitive sequences
- Mode collapse (single character dominance)

Key Insight

While attention theoretically enhances sequence modeling, in this setup it suffers from instability due to:

- Limited dataset size
- Simple architecture
- Lack of regularization

6. Comparative Analysis

Model	Strengths	Weaknesses
RNN	Stable, realistic outputs	Limited long-range memory
BLSTM	Low training loss	Fails for generation (non-causal)
Attention	Captures global context	Unstable, repetitive outputs

7. Key Learnings

- Lower training loss does **not guarantee better generation quality**
- Causality is critical in sequence generation tasks
- Attention mechanisms require careful tuning and sufficient data
- Simple models can outperform complex ones in low-data settings

8. Conclusion and Final Answer

Best Model: Vanilla RNN

Among the three models, the **Vanilla RNN** performs the best for character-level name generation on this dataset.

It achieves a strong balance between:

- Realism
- Diversity
- Novelty

It provides the most reliable and interpretable results for character-level name generation in this setting.

This study compared Vanilla RNN, BLSTM, and Attention-based RNN for character-level name generation. The results demonstrate that **model architecture must align with the generative objective** to achieve meaningful outputs.

The **Vanilla RNN** performed best overall, achieving high novelty (0.74) and diversity (0.97), while generating realistic and structured names. Despite its simplicity, it effectively captured local sequential dependencies and generalized well to unseen combinations, with only minor errors in longer sequences.

The **BLSTM model**, although achieving very low training loss, failed during generation. This is due to its **non-causal nature**, where access to future context during training creates a mismatch at inference time. Consequently, the model overfits and produces incoherent outputs, illustrating that **low training loss does not imply good generative performance**.

The **Attention-based model** introduced the ability to focus on different parts of the sequence, but exhibited instability and repetitive outputs. This indicates that attention mechanisms, while powerful, require careful tuning and sufficient data to be effective.

Overall, the results highlight the importance of **causal modeling, proper training objectives, and dataset size** in sequence generation tasks. Among the evaluated models, the Vanilla RNN provides the best balance between stability, diversity, and realism, making it the most suitable choice for this task.